

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
In [2]: dataset=pd.read_csv("CKD.csv")

In [3]: dataset
```

```
Out[3]:
```

	age	bp	sg	al	su	rbc	pc	pcc	ba	bgr	...	pcv	wc	rc	htn	dm	cad	appet	pe	ane	classification
0	2.000000	76.459948	c	3.0	0.0	normal	abnormal	notpresent	notpresent	148.112676	...	38.868902	8408.191126	4.705597	no	no	no	yes	yes	no	yes
1	3.000000	76.459948	c	2.0	0.0	normal	normal	notpresent	notpresent	148.112676	...	34.000000	12300.000000	4.705597	no	no	no	yes	poor	no	yes
2	4.000000	76.459948	a	1.0	0.0	normal	normal	notpresent	notpresent	99.000000	...	34.000000	8408.191126	4.705597	no	no	no	yes	poor	no	yes
3	5.000000	76.459948	d	1.0	0.0	normal	normal	notpresent	notpresent	148.112676	...	38.868902	8408.191126	4.705597	no	no	no	yes	poor	yes	yes
4	5.000000	50.000000	c	0.0	0.0	normal	normal	notpresent	notpresent	148.112676	...	36.000000	12400.000000	4.705597	no	no	no	yes	poor	no	yes
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
394	51.492308	70.000000	a	0.0	0.0	normal	normal	notpresent	notpresent	219.000000	...	37.000000	9800.000000	4.400000	no	no	no	yes	poor	no	yes
395	51.492308	70.000000	c	0.0	2.0	normal	normal	notpresent	notpresent	220.000000	...	27.000000	8408.191126	4.705597	yes	yes	no	yes	poor	yes	yes
396	51.492308	70.000000	c	3.0	0.0	normal	normal	notpresent	notpresent	110.000000	...	26.000000	9200.000000	3.400000	yes	yes	no	poor	poor	no	yes
397	51.492308	90.000000	a	0.0	0.0	normal	normal	notpresent	notpresent	207.000000	...	38.868902	8408.191126	4.705597	yes	yes	no	yes	poor	yes	yes
398	51.492308	80.000000	a	0.0	0.0	normal	normal	notpresent	notpresent	100.000000	...	53.000000	8500.000000	4.900000	no	no	no	yes	poor	no	no

399 rows × 25 columns

```
In [4]: dataset = dataset.drop(columns=['age'])
```

```
In [5]: dataset=pd.get_dummies(dataset,drop_first=True)
```

```
In [6]: dataset
```

```
Out[6]:
```

	bp	al	su	bgr	bu	sc	sod	pot	hrmo	pcv	...	pc_normal	pcc_present	ba_present	htn_yes	dm_yes	cad_yes	appet_yes	pe_yes	ane_yes	classification_yes
0	76.459948	3.0	0.0	148.112676	57.482105	3.077356	137.528754	4.627244	12.518156	38.868902	...	False	False	False	False	False	False	True	True	False	True
1	76.459948	2.0	0.0	148.112676	22.000000	0.700000	137.528754	4.627244	10.700000	34.000000	...	True	False	False	False	False	False	True	False	False	True
2	76.459948	1.0	0.0	99.000000	23.000000	0.600000	138.000000	4.400000	12.000000	34.000000	...	True	False	False	False	False	False	True	False	False	True
3	76.459948	1.0	0.0	148.112676	16.000000	0.700000	138.000000	3.200000	8.100000	38.868902	...	True	False	False	False	False	False	True	False	True	True
4	50.000000	0.0	0.0	148.112676	25.000000	0.600000	137.528754	4.627244	11.800000	36.000000	...	True	False	False	False	False	False	True	False	False	True
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
394	70.000000	0.0	0.0	219.000000	36.000000	1.300000	139.000000	3.700000	12.500000	37.000000	...	True	False	False	False	False	False	True	False	False	True
395	70.000000	0.0	2.0	220.000000	68.000000	2.800000	137.528754	4.627244	8.700000	27.000000	...	True	False	False	True	True	False	True	False	True	True
396	70.000000	3.0	0.0	110.000000	115.000000	6.000000	134.000000	2.700000	9.100000	26.000000	...	True	False	False	True	True	False	False	False	False	True
397	90.000000	0.0	0.0	207.000000	80.000000	6.800000	142.000000	5.500000	8.500000	38.868902	...	True	False	False	True	True	False	True	False	True	True
398	80.000000	0.0	0.0	100.000000	49.000000	1.000000	140.000000	5.000000	16.300000	53.000000	...	True	False	False	False	False	False	True	False	False	False

399 rows × 27 columns

```
In [7]: x=dataset[['bp', 'al', 'su', 'bgr', 'bu', 'sc', 'sod', 'pot', 'hrmo', 'pcv', 'wc',
'rc', 'sg_b', 'sg_c', 'sg_d', 'sg_e', 'rbc_normal', 'pc_normal',
'pcc_present', 'ba_present', 'htn_yes', 'dm_yes', 'cad_yes',
'appet_yes', 'pe_yes', 'ane_yes']]
y=dataset[['classification_yes']]
```

```
In [8]: from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=11)
```

```
In [9]: from sklearn.naive_bayes import GaussianNB
gnb = GaussianNB()
y_pred = gnb.fit(X_train, y_train).predict(X_test)
from sklearn.metrics import confusion_matrix
cm=confusion_matrix(y_test,y_pred)
print(cm)

[[22  0]
 [ 0 58]]

C:\Users\Gayathri R\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\utils\validation.py:1300: DataConversionWarning: A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples, ), for example using ravel().
  y = column_or_1d(y, warn=True)
```

```
In [11]: from sklearn.metrics import classification_report
clf_report=classification_report(y_test,y_pred)
print(clf_report)
```

	precision	recall	f1-score	support
False	1.00	1.00	1.00	22
True	1.00	1.00	1.00	58
accuracy			1.00	80
macro avg	1.00	1.00	1.00	80
weighted avg	1.00	1.00	1.00	80

```
In [13]: from sklearn.metrics import roc_auc_score
roc_auc_score(y, gnb.predict_proba(X)[:, 1])
```

```
Out[13]: 0.9965595716198126
```

```
In [14]: from sklearn.metrics import precision_score
precision=precision_score(y_test,y_pred,average='weighted')
precision
```

```
Out[14]: 1.0
```

```
In [15]: from sklearn.metrics import f1_score
f1_score(y_test,y_pred,average='weighted')
```

```
Out[15]: 1.0
```

```
In [16]: from sklearn.metrics import recall_score
recall_score(y_test,y_pred,average='weighted')
```

```
Out[16]: 1.0
```

```
In [17]: from sklearn.naive_bayes import MultinomialNB
clf = MultinomialNB()
clf.fit(X, y)
from sklearn.metrics import confusion_matrix
cm=confusion_matrix(y_test,y_pred)
print(cm)
from sklearn.metrics import classification_report
clf_report=classification_report(y_test,y_pred)
print(clf_report)

[[22  0]
 [ 0 58]]

precision    recall  f1-score   support

False        1.00        1.00        1.00        22
True         1.00        1.00        1.00        58

accuracy          1.00          1.00          1.00          80
macro avg         1.00         1.00         1.00          80
weighted avg         1.00         1.00         1.00          80

C:\Users\Gayathri R\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\utils\validation.py:1300: DataConversionWarning: A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples, ), for example using ravel().
  y = column_or_1d(y, warn=True)
```

```
In [18]: from sklearn.metrics import roc_auc_score
roc_auc_score(y, gnb.predict_proba(X)[:, 1])
```

```
Out[18]: 0.9965595716198126
```

```
In [19]: from sklearn.metrics import precision_score
precision=precision_score(y_test,y_pred,average='weighted')
precision
```

```
Out[19]: 1.0
```

```
In [20]: from sklearn.metrics import recall_score
recall_score(y_test,y_pred,average='weighted')
```

```
Out[20]: 1.0
```

```
In [21]: from sklearn.metrics import f1_score
f1_score(y_test,y_pred,average='weighted')
```

```
Out[21]: 1.0
```

```
In [22]: from sklearn.naive_bayes import ComplementNB
clf = ComplementNB()
clf.fit(X, y)
from sklearn.metrics import confusion_matrix
cm=confusion_matrix(y_test,y_pred)
print(cm)
from sklearn.metrics import classification_report
clf_report=classification_report(y_test,y_pred)
print(clf_report)

[[22  0]
 [ 0 58]]

precision    recall  f1-score   support

False        1.00        1.00        1.00        22
True         1.00        1.00        1.00        58

accuracy          1.00          1.00          1.00          80
macro avg         1.00         1.00         1.00          80
weighted avg         1.00         1.00         1.00          80

C:\Users\Gayathri R\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\utils\validation.py:1300: DataConversionWarning: A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples, ), for example using ravel().
  y = column_or_1d(y, warn=True)
```

```
In [23]: from sklearn.metrics import roc_auc_score
roc_auc_score(y, gnb.predict_proba(X)[:, 1])
```

```
Out[23]: 0.9965595716198126
```

```
In [24]: from sklearn.metrics import precision_score
precision=precision_score(y_test,y_pred,average='weighted')
precision
```

```
Out[24]: 1.0
```

```
In [25]: from sklearn.metrics import recall_score
recall_score(y_test,y_pred,average='weighted')
```

```
Out[25]: 1.0
```

```
In [26]: from sklearn.metrics import f1_score
f1_score(y_test,y_pred,average='weighted')
```

```
Out[26]: 1.0
```

```
In [27]: from sklearn.naive_bayes import BernoulliNB
clf = BernoulliNB()
clf.fit(X, y)
from sklearn.metrics import confusion_matrix
cm=confusion_matrix(y_test,y_pred)
print(cm)
from sklearn.metrics import classification_report
clf_report=classification_report(y_test,y_pred)
print(clf_report)

[[22  0]
 [ 0 58]]

precision    recall  f1-score   support

False        1.00        1.00        1.00        22
True         1.00        1.00        1.00        58

accuracy          1.00          1.00          1.00          80
macro avg         1.00         1.00         1.00          80
weighted avg         1.00         1.00         1.00          80

C:\Users\Gayathri R\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\utils\validation.py:1300: DataConversionWarning: A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples, ), for example using ravel().
  y = column_or_1d(y, warn=True)
```

```
In [28]: from sklearn.metrics import roc_auc_score
roc_auc_score(y, gnb.predict_proba(X)[:, 1])
```

```
Out[28]: 0.9965595716198126
```

```
In [29]: from sklearn.metrics import precision_score
precision=precision_score(y_test,y_pred,average='weighted')
precision
```

```
Out[29]: 1.0
```

```
In [30]: from sklearn.metrics import f1_score
f1_score(y_test,y_pred,average='weighted')
```

```
Out[30]: 1.0
```

```
In [31]: from sklearn.metrics import recall_score
recall_score(y_test,y_pred,average='weighted')
```

```
Out[31]: 1.0
```

```
In [32]: from sklearn.naive_bayes import CategoricalNB
clf = CategoricalNB()
clf.fit(X, y)
from sklearn.metrics import confusion_matrix
cm=confusion_matrix(y_test,y_pred)
print(cm)
from sklearn.metrics import classification_report
clf_report=classification_report(y_test,y_pred)
print(clf_report)

[[22  0]
 [ 0 58]]

precision    recall  f1-score   support

False        1.00        1.00        1.00        22
True         1.00        1.00        1.00        58

accuracy          1.00          1.00          1.00          80
macro avg         1.00         1.00         1.00          80
weighted avg         1.00         1.00         1.00          80

C:\Users\Gayathri R\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\utils\validation.py:1300: DataConversionWarning: A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples, ), for example using ravel().
  y = column_or_1d(y, warn=True)
```

```
In [33]: from sklearn.metrics import roc_auc_score
roc_auc_score(y, gnb.predict_proba(X)[:, 1])
```

```
Out[33]: 0.9965595716198126
```

```
In [34]: from sklearn.metrics import precision_score
precision=precision_score(y_test,y_pred,average='weighted')
precision
```

```
Out[34]: 1.0
```

```
In [35]: from sklearn.metrics import f1_score
f1_score(y_test,y_pred,average='weighted')
```

```
Out[35]: 1.0
```

```
In [36]: from sklearn.metrics import recall_score
recall_score(y_test,y_pred,average='weighted')
```



```
In [11]: import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
```

```
In [12]: dataset=pd.read_csv("CKD.csv")
```

```
In [13]: dataset
```

	age	bp	sg	al	su	rbc	pc	pcc	ba	bgr	...	pcv	wc	rc	htn	dm	cad	appet	pe	ane	classification
0	2.000000	76.459948	c	3.0	0.0	normal	abnormal	notpresent	notpresent	148.112676	...	38.868902	8408.191126	4.705597	no	no	no	yes	yes	no	yes
1	3.000000	76.459948	c	2.0	0.0	normal	normal	notpresent	notpresent	148.112676	...	34.000000	12300.000000	4.705597	no	no	no	yes	poor	no	yes
2	4.000000	76.459948	a	1.0	0.0	normal	normal	notpresent	notpresent	99.000000	...	34.000000	8408.191126	4.705597	no	no	no	yes	poor	no	yes
3	5.000000	76.459948	d	1.0	0.0	normal	normal	notpresent	notpresent	148.112676	...	38.868902	8408.191126	4.705597	no	no	no	yes	poor	yes	yes
4	5.000000	50.000000	c	0.0	0.0	normal	normal	notpresent	notpresent	148.112676	...	36.000000	12400.000000	4.705597	no	no	no	yes	poor	no	yes
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
394	51.492308	70.000000	a	0.0	0.0	normal	normal	notpresent	notpresent	219.000000	...	37.000000	9800.000000	4.400000	no	no	no	yes	poor	no	yes
395	51.492308	70.000000	c	0.0	2.0	normal	normal	notpresent	notpresent	220.000000	...	27.000000	8408.191126	4.705597	yes	yes	no	yes	poor	yes	yes
396	51.492308	70.000000	c	3.0	0.0	normal	normal	notpresent	notpresent	110.000000	...	26.000000	9200.000000	3.400000	yes	yes	no	poor	poor	no	yes
397	51.492308	90.000000	a	0.0	0.0	normal	normal	notpresent	notpresent	207.000000	...	38.868902	8408.191126	4.705597	yes	yes	no	yes	poor	yes	yes
398	51.492308	80.000000	a	0.0	0.0	normal	normal	notpresent	notpresent	100.000000	...	53.000000	8500.000000	4.900000	no	no	no	yes	poor	no	no

399 rows × 25 columns

```
In [14]: dataset = dataset.drop(columns=['age'])
```

```
In [15]: dataset=pd.get_dummies(dataset,drop_first=True)
```

```
In [16]: dataset
```

	bp	al	su	bgr	bu	sc	sod	pot	hrmo	pcv	...	pc_normal	pcc_present	ba_present	htn_yes	dm_yes	cad_yes	appet_yes	pe_yes	ane_yes	classification_yes
0	76.459948	3.0	0.0	148.112676	57.482105	3.077356	137.528754	4.627244	12.518156	38.868902	...	False	False	False	False	False	False	True	True	False	True
1	76.459948	2.0	0.0	148.112676	22.000000	0.700000	137.528754	4.627244	10.700000	34.000000	...	True	False	False	False	False	False	True	False	False	True
2	76.459948	1.0	0.0	99.000000	23.000000	0.600000	138.000000	4.400000	12.000000	34.000000	...	True	False	False	False	False	False	True	False	False	True
3	76.459948	1.0	0.0	148.112676	16.000000	0.700000	138.000000	3.200000	8.100000	38.868902	...	True	False	False	False	False	False	True	False	True	True
4	50.000000	0.0	0.0	148.112676	25.000000	0.600000	137.528754	4.627244	11.800000	36.000000	...	True	False	False	False	False	False	True	False	False	True
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
394	70.000000	0.0	0.0	219.000000	36.000000	1.300000	139.000000	3.700000	12.500000	37.000000	...	True	False	False	False	False	False	True	False	False	True
395	70.000000	0.0	2.0	220.000000	68.000000	2.800000	137.528754	4.627244	8.700000	27.000000	...	True	False	False	True	True	False	True	False	True	True
396	70.000000	3.0	0.0	110.000000	115.000000	6.000000	134.000000	2.700000	9.100000	26.000000	...	True	False	False	True	True	False	False	False	False	True
397	90.000000	0.0	0.0	207.000000	80.000000	6.800000	142.000000	5.500000	8.500000	38.868902	...	True	False	False	True	True	False	True	False	True	True
398	80.000000	0.0	0.0	100.000000	49.000000	1.000000	140.000000	5.000000	16.300000	53.000000	...	True	False	False	False	False	False	True	False	False	False

399 rows × 27 columns

```
In [17]: x=dataset[['bp', 'al', 'su', 'bgr', 'bu', 'sc', 'sod', 'pot', 'hrmo', 'pcv', 'wc',
                'rc', 'sg_b', 'sg_c', 'sg_d', 'sg_e', 'rbc_normal', 'pc_normal',
                'pcc_present', 'ba_present', 'htn_yes', 'dm_yes', 'cad_yes',
                'appet_yes', 'pe_yes', 'ane_yes']]
y=dataset[['classification_yes']]
```

```
In [18]: from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=11)
```

```
In [19]: from sklearn.ensemble import RandomForestClassifier
classifier = RandomForestClassifier(random_state=0, n_estimators=100,criterion='entropy')
classifier.fit(x, y)
```

C:\Users\Gayathri R\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\base.py:1474: DataConversionWarning: A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().

RandomForestClassifier

RandomForestClassifier(criterion='entropy', random_state=0)

```
In [29]: y_pred=classifier.predict(x_test)
from sklearn.metrics import confusion_matrix
cm=confusion_matrix(y_test,y_pred)
print(cm)
```

```
[[22  0]
 [ 0 58]]
```

```
In [30]: from sklearn.metrics import classification_report
clf_report=classification_report(y_test,y_pred)
print(clf_report)
```

	precision	recall	f1-score	support
False	1.00	1.00	1.00	22
True	1.00	1.00	1.00	58
accuracy			1.00	80
macro avg	1.00	1.00	1.00	80
weighted avg	1.00	1.00	1.00	80

```
In [31]: from sklearn.metrics import roc_auc_score
roc_auc_score(y, classifier.predict_proba(x)[:, 1])
```

Out[31]: 1.0

```
In [32]: from sklearn.metrics import precision_score
precision=precision_score(y_test,y_pred,average='weighted')
precision
```

Out[32]: 1.0

```
In [33]: from sklearn.metrics import f1_score
f1_score(y_test,y_pred,average='weighted')
```

Out[33]: 1.0

```
In [34]: from sklearn.metrics import recall_score
recall_score(y_test,y_pred,average='weighted')
```

Out[34]: 1.0


```
In [17]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
In [18]: dataset=pd.read_csv("CKD.csv")
```

```
In [19]: dataset
```

Out[19]:

	age	bp	sg	al	su	rbc	pc	pcc	ba	bgr	...	pcv	wc	rc	htn	dm	cad	appet	pe	ane	classification
0	2.000000	76.459948	c	3.0	0.0	normal	abnormal	notpresent	notpresent	148.112676	...	38.868902	8408.191126	4.705597	no	no	no	yes	yes	no	yes
1	3.000000	76.459948	c	2.0	0.0	normal	normal	notpresent	notpresent	148.112676	...	34.000000	12300.000000	4.705597	no	no	no	yes	poor	no	yes
2	4.000000	76.459948	a	1.0	0.0	normal	normal	notpresent	notpresent	99.000000	...	34.000000	8408.191126	4.705597	no	no	no	yes	poor	no	yes
3	5.000000	76.459948	d	1.0	0.0	normal	normal	notpresent	notpresent	148.112676	...	38.868902	8408.191126	4.705597	no	no	no	yes	poor	yes	yes
4	5.000000	50.000000	c	0.0	0.0	normal	normal	notpresent	notpresent	148.112676	...	36.000000	12400.000000	4.705597	no	no	no	yes	poor	no	yes
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
394	51.492308	70.000000	a	0.0	0.0	normal	normal	notpresent	notpresent	219.000000	...	37.000000	9800.000000	4.400000	no	no	no	yes	poor	no	yes
395	51.492308	70.000000	c	0.0	2.0	normal	normal	notpresent	notpresent	220.000000	...	27.000000	8408.191126	4.705597	yes	yes	no	yes	poor	yes	yes
396	51.492308	70.000000	c	3.0	0.0	normal	normal	notpresent	notpresent	110.000000	...	26.000000	9200.000000	3.400000	yes	yes	no	poor	poor	no	yes
397	51.492308	90.000000	a	0.0	0.0	normal	normal	notpresent	notpresent	207.000000	...	38.868902	8408.191126	4.705597	yes	yes	no	yes	poor	yes	yes
398	51.492308	80.000000	a	0.0	0.0	normal	normal	notpresent	notpresent	100.000000	...	53.000000	8500.000000	4.900000	no	no	no	yes	poor	no	no

399 rows × 25 columns

```
In [20]: dataset = dataset.drop(columns=['age'])
```

```
In [21]: dataset=pd.get_dummies(dataset,drop_first=True)
```

```
In [22]: dataset
```

Out[22]:

	bp	al	su	bgr	bu	sc	sod	pot	hrmo	pcv	...	pc_normal	pcc_present	ba_present	htn_yes	dm_yes	cad_yes	appet_yes	pe_yes	ane_yes	classification_yes
0	76.459948	3.0	0.0	148.112676	57.482105	3.077356	137.528754	4.627244	12.518156	38.868902	...	False	False	False	False	False	False	True	True	False	True
1	76.459948	2.0	0.0	148.112676	22.000000	0.700000	137.528754	4.627244	10.700000	34.000000	...	True	False	False	False	False	False	True	False	False	True
2	76.459948	1.0	0.0	99.000000	23.000000	0.600000	138.000000	4.400000	12.000000	34.000000	...	True	False	False	False	False	False	True	False	False	True
3	76.459948	1.0	0.0	148.112676	16.000000	0.700000	138.000000	3.200000	8.100000	38.868902	...	True	False	False	False	False	False	True	False	True	True
4	50.000000	0.0	0.0	148.112676	25.000000	0.600000	137.528754	4.627244	11.800000	36.000000	...	True	False	False	False	False	False	True	False	False	True
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
394	70.000000	0.0	0.0	219.000000	36.000000	1.300000	139.000000	3.700000	12.500000	37.000000	...	True	False	False	False	False	False	True	False	False	True
395	70.000000	0.0	2.0	220.000000	68.000000	2.800000	137.528754	4.627244	8.700000	27.000000	...	True	False	False	True	True	False	True	False	True	True
396	70.000000	3.0	0.0	110.000000	115.000000	6.000000	134.000000	2.700000	9.100000	26.000000	...	True	False	False	True	True	False	False	False	False	True
397	90.000000	0.0	0.0	207.000000	80.000000	6.800000	142.000000	5.500000	8.500000	38.868902	...	True	False	False	True	True	False	True	False	True	True
398	80.000000	0.0	0.0	100.000000	49.000000	1.000000	140.000000	5.000000	16.300000	53.000000	...	True	False	False	False	False	False	True	False	False	False

399 rows × 27 columns

```
In [23]: x=dataset[['bp', 'al', 'su', 'bgr', 'bu', 'sc', 'sod', 'pot', 'hrmo', 'pcv', 'wc',
                'rc', 'sg_b', 'sg_c', 'sg_d', 'sg_e', 'rbc_normal', 'pc_normal',
                'pcc_present', 'ba_present', 'htn_yes', 'dm_yes', 'cad_yes',
                'appet_yes', 'pe_yes', 'ane_yes']]
y=dataset[['classification_yes']]
```

```
In [24]: from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=11)
```

```
In [25]: from sklearn.svm import SVC
clf=SVC(kernel="poly",C=100)
clf.fit(x_train,y_train)

C:\Users\Gayathri R\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\utils\validation.py:1300: DataConversionWarning: A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_sample
s, ), for example using ravel().
  y = column_or_1d(y, warn=True)
```

Out[25]:

SVC

SVC(C=100, kernel='poly')

```
In [26]: y_pred=clf.predict(x_test)
from sklearn.metrics import confusion_matrix
cm=confusion_matrix(y_test,y_pred)
print(cm)

[[17  5]
 [22 36]]
```

```
In [27]: from sklearn.metrics import classification_report
clf_report=classification_report(y_test,y_pred)
print(clf_report)

              precision    recall  f1-score   support

     False       0.44       0.77       0.56         22
     True        0.88       0.62       0.73         58

 accuracy              0.66         80
 macro avg           0.66       0.70       0.64         80
weighted avg           0.76       0.66       0.68         80
```

```
In [34]: from sklearn.metrics import precision_score
precision=precision_score(y_test,y_pred,average='weighted')
precision
```

Out[34]: 0.7564571607254534

```
In [35]: from sklearn.metrics import f1_score
f1_score(y_test,y_pred,average='weighted')
```

Out[35]: 0.6805514157973174

```
In [36]: from sklearn.metrics import recall_score
recall_score(y_test,y_pred,average='weighted')
```


In [2]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

In [3]:

```
dataset=pd.read_csv("CKD.csv")
```

In [4]:

```
dataset
```

Out[4]:

	age	bp	sg	al	su	rbc	pc	pcc	ba	bgr	...	
0	2.000000	76.459948	c	3.0	0.0	normal	abnormal	notpresent	notpresent	148.112676	...	38.868
1	3.000000	76.459948	c	2.0	0.0	normal	normal	notpresent	notpresent	148.112676	...	34.000
2	4.000000	76.459948	a	1.0	0.0	normal	normal	notpresent	notpresent	99.000000	...	34.000
3	5.000000	76.459948	d	1.0	0.0	normal	normal	notpresent	notpresent	148.112676	...	38.868
4	5.000000	50.000000	c	0.0	0.0	normal	normal	notpresent	notpresent	148.112676	...	36.000
...	...	...	...	...	...	...	...	...	...	...	...	...
394	51.492308	70.000000	a	0.0	0.0	normal	normal	notpresent	notpresent	219.000000	...	37.000
395	51.492308	70.000000	c	0.0	2.0	normal	normal	notpresent	notpresent	220.000000	...	27.000
396	51.492308	70.000000	c	3.0	0.0	normal	normal	notpresent	notpresent	110.000000	...	26.000
397	51.492308	90.000000	a	0.0	0.0	normal	normal	notpresent	notpresent	207.000000	...	38.868
398	51.492308	80.000000	a	0.0	0.0	normal	normal	notpresent	notpresent	100.000000	...	53.000

399 rows × 25 columns

In [5]:

```
dataset=pd.get_dummies(dataset,drop_first=True)
```

In [6]:

```
dataset
```

Out[6]:

	age	bp	al	su	bgr	bu	sc	sod	pot	hrmo
0	2.000000	76.459948	3.0	0.0	148.112676	57.482105	3.077356	137.528754	4.627244	12.518156
1	3.000000	76.459948	2.0	0.0	148.112676	22.000000	0.700000	137.528754	4.627244	10.700000
2	4.000000	76.459948	1.0	0.0	99.000000	23.000000	0.600000	138.000000	4.400000	12.000000
3	5.000000	76.459948	1.0	0.0	148.112676	16.000000	0.700000	138.000000	3.200000	8.100000
4	5.000000	50.000000	0.0	0.0	148.112676	25.000000	0.600000	137.528754	4.627244	11.800000
...	...	...	...	...	...	...	...	...	...	...
394	51.492308	70.000000	0.0	0.0	219.000000	36.000000	1.300000	139.000000	3.700000	12.500000
395	51.492308	70.000000	0.0	2.0	220.000000	68.000000	2.800000	137.528754	4.627244	8.700000

	age	bp	al	su	bgr	bu	sc	sod	pot	hrmo
396	51.492308	70.000000	3.0	0.0	110.000000	115.000000	6.000000	134.000000	2.700000	9.100000
397	51.492308	90.000000	0.0	0.0	207.000000	80.000000	6.800000	142.000000	5.500000	8.500000
398	51.492308	80.000000	0.0	0.0	100.000000	49.000000	1.000000	140.000000	5.000000	16.300000

399 rows × 28 columns

In [7]:

```
dataset = dataset.drop(columns=['age'])
```

In [8]:

```
dataset
```

Out[8]:

	bp	al	su	bgr	bu	sc	sod	pot	hrmo	pcv
0	76.459948	3.0	0.0	148.112676	57.482105	3.077356	137.528754	4.627244	12.518156	38.868902
1	76.459948	2.0	0.0	148.112676	22.000000	0.700000	137.528754	4.627244	10.700000	34.000000
2	76.459948	1.0	0.0	99.000000	23.000000	0.600000	138.000000	4.400000	12.000000	34.000000
3	76.459948	1.0	0.0	148.112676	16.000000	0.700000	138.000000	3.200000	8.100000	38.868902
4	50.000000	0.0	0.0	148.112676	25.000000	0.600000	137.528754	4.627244	11.800000	36.000000
...	...	...	...	...	...	...	...	...	...	...
394	70.000000	0.0	0.0	219.000000	36.000000	1.300000	139.000000	3.700000	12.500000	37.000000
395	70.000000	0.0	2.0	220.000000	68.000000	2.800000	137.528754	4.627244	8.700000	27.000000
396	70.000000	3.0	0.0	110.000000	115.000000	6.000000	134.000000	2.700000	9.100000	26.000000
397	90.000000	0.0	0.0	207.000000	80.000000	6.800000	142.000000	5.500000	8.500000	38.868902
398	80.000000	0.0	0.0	100.000000	49.000000	1.000000	140.000000	5.000000	16.300000	53.000000

399 rows × 27 columns

In [9]:

```
dataset.columns
```

Out[9]:

```
Index(['bp', 'al', 'su', 'bgr', 'bu', 'sc', 'sod', 'pot', 'hrmo', 'pcv', 'wc',
      'rc', 'sg_b', 'sg_c', 'sg_d', 'sg_e', 'rbc_normal', 'pc_normal',
      'pcc_present', 'ba_present', 'htn_yes', 'dm_yes', 'cad_yes',
      'appet_yes', 'pe_yes', 'ane_yes', 'classification_yes'],
      dtype='object')
```

In [10]:

```
x=dataset[['bp', 'al', 'su', 'bgr', 'bu', 'sc', 'sod', 'pot', 'hrmo', 'pcv', 'wc',
          'rc', 'sg_b', 'sg_c', 'sg_d', 'sg_e', 'rbc_normal', 'pc_normal',
          'pcc_present', 'ba_present', 'htn_yes', 'dm_yes', 'cad_yes',
          'appet_yes', 'pe_yes', 'ane_yes']]
y=dataset[['classification_yes']]
```

In [11]:

```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=11)
```

In [12]:

```
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
x_train=sc.fit_transform(x_train)
x_test=sc.transform(x_test)
```

In [13]:

```
from sklearn.linear_model import LogisticRegression
clf = LogisticRegression(random_state=0).fit(x, y)
```

C:\Users\Gayathri R\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\utils\validation.py:1300: DataConversionWarning: A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().

```
y = column_or_1d(y, warn=True)
```

C:\Users\Gayathri R\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\linear_model_logistic.py:469: ConvergenceWarning: lbfgs failed to converge (status=1): STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

In [14]:

```
y_pred=clf.predict(x_test)
from sklearn.metrics import confusion_matrix
cm=confusion_matrix(y_test,y_pred)
print(cm)
from sklearn.metrics import classification_report
clf_report=classification_report(y_test,y_pred)
print(clf_report)
```

[[22 0]

[6 52]]

	precision	recall	f1-score	support
False	0.79	1.00	0.88	22
True	1.00	0.90	0.95	58
accuracy			0.93	80
macro avg	0.89	0.95	0.91	80
weighted avg	0.94	0.93	0.93	80

C:\Users\Gayathri R\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\base.py:493: UserWarning: X does not have valid feature names, but LogisticRegression was fitted with feature names

```
warnings.warn(
```

In [16]:

```
from sklearn.metrics import roc_auc_score
roc_auc_score(y, clf.predict_proba(x)[: , 1])
```

Out[16]:
0.9797054886211511

In [17]:

```
from sklearn.metrics import precision_score
precision=precision_score(y_test,y_pred,average='weighted')
precision
```

Out[17]:
0.9410714285714284

In [18]:

```
from sklearn.metrics import f1_score
f1_score(y_test,y_pred,average='weighted')
```

Out[18]:
0.9274545454545453

In [19]:

```
from sklearn.metrics import recall_score
recall_score(y_test,y_pred,average='weighted')
```

Out[19]:
0.925

In []:

In [1]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

In [2]:

```
dataset=pd.read_csv("CKD.csv")
```

In [3]:

```
dataset
```

Out[3]:

	age	bp	sg	al	su	rbc	pc	pcc	ba	bgr	...	
0	2.000000	76.459948	c	3.0	0.0	normal	abnormal	notpresent	notpresent	148.112676	...	38.868
1	3.000000	76.459948	c	2.0	0.0	normal	normal	notpresent	notpresent	148.112676	...	34.000
2	4.000000	76.459948	a	1.0	0.0	normal	normal	notpresent	notpresent	99.000000	...	34.000
3	5.000000	76.459948	d	1.0	0.0	normal	normal	notpresent	notpresent	148.112676	...	38.868
4	5.000000	50.000000	c	0.0	0.0	normal	normal	notpresent	notpresent	148.112676	...	36.000
...	...	...	...	...	...	...	...	...	...	...	...	...
394	51.492308	70.000000	a	0.0	0.0	normal	normal	notpresent	notpresent	219.000000	...	37.000
395	51.492308	70.000000	c	0.0	2.0	normal	normal	notpresent	notpresent	220.000000	...	27.000
396	51.492308	70.000000	c	3.0	0.0	normal	normal	notpresent	notpresent	110.000000	...	26.000
397	51.492308	90.000000	a	0.0	0.0	normal	normal	notpresent	notpresent	207.000000	...	38.868
398	51.492308	80.000000	a	0.0	0.0	normal	normal	notpresent	notpresent	100.000000	...	53.000

399 rows × 25 columns

In [4]:

```
dataset = dataset.drop(columns=['age'])
```

In [5]:

```
dataset=pd.get_dummies(dataset,drop_first=True)
```

In [6]:

```
dataset
```

Out[6]:

	bp	al	su	bgr	bu	sc	sod	pot	hrmo	pcv
0	76.459948	3.0	0.0	148.112676	57.482105	3.077356	137.528754	4.627244	12.518156	38.868902
1	76.459948	2.0	0.0	148.112676	22.000000	0.700000	137.528754	4.627244	10.700000	34.000000
2	76.459948	1.0	0.0	99.000000	23.000000	0.600000	138.000000	4.400000	12.000000	34.000000
3	76.459948	1.0	0.0	148.112676	16.000000	0.700000	138.000000	3.200000	8.100000	38.868902
4	50.000000	0.0	0.0	148.112676	25.000000	0.600000	137.528754	4.627244	11.800000	36.000000
...	...	...	...	...	...	...	...	...	...	...

	bp	al	su	bgr	bu	sc	sod	pot	hrmo	pcv
394	70.000000	0.0	0.0	219.000000	36.000000	1.300000	139.000000	3.700000	12.500000	37.000000
395	70.000000	0.0	2.0	220.000000	68.000000	2.800000	137.528754	4.627244	8.700000	27.000000
396	70.000000	3.0	0.0	110.000000	115.000000	6.000000	134.000000	2.700000	9.100000	26.000000
397	90.000000	0.0	0.0	207.000000	80.000000	6.800000	142.000000	5.500000	8.500000	38.868902
398	80.000000	0.0	0.0	100.000000	49.000000	1.000000	140.000000	5.000000	16.300000	53.000000

399 rows × 27 columns

In [7]:

```
x=dataset[['bp', 'al', 'su', 'bgr', 'bu', 'sc', 'sod', 'pot', 'hrmo', 'pcv', 'wc',
            'rc', 'sg_b', 'sg_c', 'sg_d', 'sg_e', 'rbc_normal', 'pc_normal',
            'pcc_present', 'ba_present', 'htn_yes', 'dm_yes', 'cad_yes',
            'appet_yes', 'pe_yes', 'ane_yes']]
y=dataset[['classification_yes']]
```

In [8]:

```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=11)
```

In [9]:

```
from sklearn.tree import DecisionTreeClassifier
clf = DecisionTreeClassifier(random_state=100)
clf.fit(x, y)
```

Out[9]:

```
DecisionTreeClassifier
DecisionTreeClassifier(random_state=100)
```

In [10]:

```
y_pred=clf.predict(x_test)
from sklearn.metrics import confusion_matrix
cm=confusion_matrix(y_test,y_pred)
print(cm)
from sklearn.metrics import classification_report
clf_report=classification_report(y_test,y_pred)
print(clf_report)
```

```
[[22  0]
 [ 0 58]]
```

	precision	recall	f1-score	support
False	1.00	1.00	1.00	22
True	1.00	1.00	1.00	58
accuracy			1.00	80
macro avg	1.00	1.00	1.00	80
weighted avg	1.00	1.00	1.00	80

In [11]:

```
from sklearn.metrics import roc_auc_score
```

```
roc_auc_score(y, clf.predict_proba(x)[: , 1])
```

Out[11]:

1.0

In [12]:

```
from sklearn.metrics import precision_score
precision=precision_score(y_test,y_pred,average='weighted')
precision
```

Out[12]:

1.0

In [13]:

```
from sklearn.metrics import f1_score
f1_score(y_test,y_pred,average='weighted')
```

Out[13]:

1.0

In [14]:

```
from sklearn.metrics import recall_score
recall_score(y_test,y_pred,average='weighted')
```

Out[14]:

1.0

In []:

In [1]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

In [2]:

```
dataset=pd.read_csv("CKD.csv")
```

In [3]:

```
dataset
```

Out[3]:

	age	bp	sg	al	su	rbc	pc	pcc	ba	bgr	...	
0	2.000000	76.459948	c	3.0	0.0	normal	abnormal	notpresent	notpresent	148.112676	...	38.868
1	3.000000	76.459948	c	2.0	0.0	normal	normal	notpresent	notpresent	148.112676	...	34.000
2	4.000000	76.459948	a	1.0	0.0	normal	normal	notpresent	notpresent	99.000000	...	34.000
3	5.000000	76.459948	d	1.0	0.0	normal	normal	notpresent	notpresent	148.112676	...	38.868
4	5.000000	50.000000	c	0.0	0.0	normal	normal	notpresent	notpresent	148.112676	...	36.000
...	...	...	...	...	...	...	...	...	...	...	...	...
394	51.492308	70.000000	a	0.0	0.0	normal	normal	notpresent	notpresent	219.000000	...	37.000
395	51.492308	70.000000	c	0.0	2.0	normal	normal	notpresent	notpresent	220.000000	...	27.000
396	51.492308	70.000000	c	3.0	0.0	normal	normal	notpresent	notpresent	110.000000	...	26.000
397	51.492308	90.000000	a	0.0	0.0	normal	normal	notpresent	notpresent	207.000000	...	38.868
398	51.492308	80.000000	a	0.0	0.0	normal	normal	notpresent	notpresent	100.000000	...	53.000

399 rows × 25 columns

In [4]:

```
dataset=pd.get_dummies(dataset,drop_first=True)
```

In [5]:

```
dataset = dataset.drop(columns=['age'])
```

In [6]:

```
dataset
```

Out[6]:

	bp	al	su	bgr	bu	sc	sod	pot	hrmo	pcv
0	76.459948	3.0	0.0	148.112676	57.482105	3.077356	137.528754	4.627244	12.518156	38.868902
1	76.459948	2.0	0.0	148.112676	22.000000	0.700000	137.528754	4.627244	10.700000	34.000000
2	76.459948	1.0	0.0	99.000000	23.000000	0.600000	138.000000	4.400000	12.000000	34.000000
3	76.459948	1.0	0.0	148.112676	16.000000	0.700000	138.000000	3.200000	8.100000	38.868902
4	50.000000	0.0	0.0	148.112676	25.000000	0.600000	137.528754	4.627244	11.800000	36.000000
...	...	...	...	...	...	...	...	...	...	...

	bp	al	su	bgr	bu	sc	sod	pot	hrmo	pcv
394	70.000000	0.0	0.0	219.000000	36.000000	1.300000	139.000000	3.700000	12.500000	37.000000
395	70.000000	0.0	2.0	220.000000	68.000000	2.800000	137.528754	4.627244	8.700000	27.000000
396	70.000000	3.0	0.0	110.000000	115.000000	6.000000	134.000000	2.700000	9.100000	26.000000
397	90.000000	0.0	0.0	207.000000	80.000000	6.800000	142.000000	5.500000	8.500000	38.868902
398	80.000000	0.0	0.0	100.000000	49.000000	1.000000	140.000000	5.000000	16.300000	53.000000

399 rows × 27 columns

In [7]:

```
dataset.columns
```

Out[7]:

```
Index(['bp', 'al', 'su', 'bgr', 'bu', 'sc', 'sod', 'pot', 'hrmo', 'pcv', 'wc',
      'rc', 'sg_b', 'sg_c', 'sg_d', 'sg_e', 'rbc_normal', 'pc_normal',
      'pcc_present', 'ba_present', 'htn_yes', 'dm_yes', 'cad_yes',
      'appet_yes', 'pe_yes', 'ane_yes', 'classification_yes'],
      dtype='object')
```

In [8]:

```
x=dataset[['bp', 'al', 'su', 'bgr', 'bu', 'sc', 'sod', 'pot', 'hrmo', 'pcv', 'wc',
          'rc', 'sg_b', 'sg_c', 'sg_d', 'sg_e', 'rbc_normal', 'pc_normal',
          'pcc_present', 'ba_present', 'htn_yes', 'dm_yes', 'cad_yes',
          'appet_yes', 'pe_yes', 'ane_yes']]
y=dataset[['classification_yes']]
```

In [9]:

```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=11)
```

In [10]:

```
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
x_train=sc.fit_transform(x_train)
x_test=sc.transform(x_test)
```

In [11]:

```
from sklearn.neighbors import KNeighborsClassifier
neigh = KNeighborsClassifier(n_neighbors=3)
neigh.fit(x, y)
```

C:\Users\Gayathri R\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\neighbors_classification.py:238: DataConversionWarning: A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().

```
return self._fit(X, y)
```

Out[11]:

```
KNeighborsClassifier
KNeighborsClassifier(n_neighbors=3)
```

In [12]:

```

y_pred=neigh.predict(x_test)
from sklearn.metrics import confusion_matrix
cm=confusion_matrix(y_test,y_pred)
print(cm)
from sklearn.metrics import classification_report
clf_report=classification_report(y_test,y_pred)
print(clf_report)

```

C:\Users\Gayathri R\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\base.py:493: UserWarning: X does not have valid feature names, but KNeighborsClassifier was fitted with feature names

```
warnings.warn(
```

C:\Users\Gayathri R\AppData\Local\Programs\Python\Python312\Lib\site-packages\joblib\externals\loky\backend\context.py:136: UserWarning: Could not find the number of physical cores for the following reason:

[WinError 2] The system cannot find the file specified

Returning the number of logical cores instead. You can silence this warning by setting LOKY_MAX_CPU_COUNT to the number of cores you want to use.

```
warnings.warn(
```

File "C:\Users\Gayathri R\AppData\Local\Programs\Python\Python312\Lib\site-packages\joblib\externals\loky\backend\context.py", line 257, in _count_physical_cores

```
cpu_info = subprocess.run(
    ^^^^^^^^^^^^^^^^^
```

File "C:\Users\Gayathri R\AppData\Local\Programs\Python\Python312\Lib\subprocess.py", line 548, in run

```
with Popen(*popenargs, **kwargs) as process:
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

File "C:\Users\Gayathri R\AppData\Local\Programs\Python\Python312\Lib\subprocess.py", line 1026, in __init__

```
self._execute_child(args, executable, preexec_fn, close_fds,
```

File "C:\Users\Gayathri R\AppData\Local\Programs\Python\Python312\Lib\subprocess.py", line 1538, in _execute_child

```
hp, ht, pid, tid = _winapi.CreateProcess(executable, args,
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
[[ 0 22]
 [ 0 58]]
```

	precision	recall	f1-score	support
False	0.00	0.00	0.00	22
True	0.72	1.00	0.84	58
accuracy			0.72	80
macro avg	0.36	0.50	0.42	80
weighted avg	0.53	0.72	0.61	80

C:\Users\Gayathri R\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\metrics_classification.py:1509: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

```
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

C:\Users\Gayathri R\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\metrics_classification.py:1509: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

```
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

C:\Users\Gayathri R\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\metrics_classification.py:1509: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

g set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

```
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

In [14]:

```
from sklearn.metrics import roc_auc_score
roc_auc_score(y, neigh.predict_proba(x)[: , 1])
```

Out[14]:

0.9649799196787148

In [15]:

```
from sklearn.metrics import precision_score
precision=precision_score(y_test,y_pred,average='weighted')
precision
```

C:\Users\Gayathri R\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\metrics_classification.py:1509: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

```
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

Out[15]:

0.525625

In [16]:

```
from sklearn.metrics import f1_score
f1_score(y_test,y_pred,average='weighted')
```

Out[16]:

0.6094202898550725

In [17]:

```
from sklearn.metrics import recall_score
recall_score(y_test,y_pred,average='weighted')
```

Out[17]:

0.725

In []: